COMPLETE COURSE NOTES

JavaScript

From Fundamentals to Advanced Concepts

DOM • Events • OOP • Async • BOM • ES6+

Days 1 – 98 | 41 Sessions | Complete Class Notes

Table of Contents

Day 1	JavaScript Introduction	Day 2	Client-Side JavaScript & DOM
Day 3	DOM Access Methods & Output	Day 4	Output & Input Techniques
Day 5	Practical: Modal & Functions	Day 6	var, let, const & Hoisting
Day 7	Variables, Datatypes & JSON	Day 8	Number Type & Conversions
Day 9-10	Template Literals & Strings	Day 11	String Methods – Part 1
Day 12	String Methods – Part 2	Day 13	match(), Boolean, Operators
Day 14	Conditional Statements	Day 15	switch / case / default
Day 17	Non-Primitive Types & Arrays	Day 18	Array Methods – map, forEach
Day 19	Array – Insert, Delete, Sort	Day 20	filter() and reduce()
Day 21	fetch(), async / await	Day 22	Objects & API Integration
Day 23	findIndex & Shopping Cart	Day 24	Map, Symbol & Date
Day 25	setTimeout, setInterval, Debouncing	Day 26	Functions, Callbacks & Promises
Day 27	Arrow Functions vs Expressions	Day 28	Rest, Spread & Generators
Day 29	Higher Order Functions	Day 30	Unary Operators & Async Examples
Day 31	Project: KFC Order App	Day 32	Login & Nested Lists
Day 33	Loops, Jump Statements & React Intro	Day 34	instanceof, Exception Handling
Day 35	OOP – Classes & Constructors	Day 35b	Classes – Setter, Getter, Modules
Day 36	Aggregation & Inheritance	Day 37	JavaScript Browser Events
Day 38	Mouse, Keyboard & Clipboard Events	Day 39	Form Events & Quiz App Project
Day 98	BOM – Browser Object Model		

DAY 1

JavaScript Introduction

21 April 2025

What is JavaScript?

JavaScript is a **lightweight, Interpreted and Just-in-Time (JIT) Compiled** programming language.

- **Light weight** — refers to memory occupied and how heavy the application is.
- **Interpreted** — line-by-line translation at runtime.
- **Compiled** — translating the entire program at once.
  - **JIT (Just-in-Time)** — JavaScript is loaded into the browser and compiled there, when the user sends the request.
  - **AOT (Ahead-of-Time)** — compiled and processed at application level even before the request arrives.

Engines and Compilers

- Ivy
- Babel
- Node
- V8

Programming Paradigms Supported

- Structural Programming
- Functional Programming
- Imperative Programming
- Object Oriented Programming

> **Note:** JavaScript is NOT a pure OOP language. It supports only a few features of OOP.

Where JavaScript is Used

Layer	Technology
Client Side	JavaScript with HTML
Server Side	Node.js

Database	MongoDB
Animation Tools	Flash, 3DS Max

Full Stack (MERN):
Frontend: React [JS] → Backend: Node.js + Express.js → Database: MongoDB [JS]

Issues with JavaScript

- **Not strongly typed** — a variable can hold any type at any time.
- **Not strictly typed** — variables can be used without declaration (unless `"use strict"` is enabled).
- **No uniform data control** — objects can have inconsistent keys.
- **Security concerns** — JavaScript can be blocked by the browser.

DAY 2

Client-Side JavaScript & DOM

22 April 2025

Purpose of Client-Side JavaScript

- Avoids burden on the server.
- Improves application performance by managing interactions on the client side.

Client-Side Interactions

a. DOM Manipulations

- Adding elements into the page
- Removing elements from the page
- Updating data in elements
- Configuring styles to elements dynamically
- Configuring events to elements dynamically

b. Validations

Verifying user input before sending to the server.

c. Client-Side Device Management

- Managing client memory (username/password in cache)
- Managing client devices (e.g., printing offline)
- Managing user location (geolocation)
- Sharing data with other applications

Important FAQs

Q: What is the MIME type for a script?

`type='text/javascript'` — bulk loading
`type='text/module'` — module-based loading (higher performance)

Q: Difference between script in <head> and <body>?

head — script loads into browser memory first, then used in the page.
body — script loads directly into the page; not part of browser memory.

Q: How does JavaScript convert Static DOM to Dynamic DOM?

HTML → HTML parser → Static DOM
CSS → CSS parser → CSSOM (styles the static DOM)
JS → JavaScript engine → uses Static DOM + **events and functions** → Dynamic DOM

Q: What is strict mode?

Prevents use of variables without declaration.

```
"use strict";
a = 10;
console.log(a); // ReferenceError: a is not defined
```

Q: How to target JavaScript for Legacy browsers?

Enclose code in HTML comments inside the script tag.

Q: How to link an external JavaScript file to HTML?

`<script src="filename.js"></script>`

DAY 3

DOM Access Methods & Output Operations

25 April 2025

Methods to Access DOM Elements

Method	Returns	Type
`getElementById()`	HTMLElement | null	Static
`getElementsByClassName()`	HTMLCollection[]	Dynamic (Live)
`getElementsByTagName()`	HTMLCollection[]	Dynamic (Live)
`getElementsByName()`	NodeList	Dynamic
`querySelector()`	HTMLElement | null	Static
`querySelectorAll()`	NodeList[]	Static

Static vs Dynamic DOM Access

Static DOM Access — Snapshot taken at the time of access. If DOM changes later, the reference does NOT update.

Dynamic (Live) DOM Access — Automatically reflects changes. If elements are added or removed, the collection updates itself.

Rule: querySelector | querySelectorAll | getElementById → Static DOM only
getElementsByClassName | getElementsByTagName | getElementsByName → Live (Dynamic) DOM

Converting HTMLCollection to Array

```
// Method 1
const arr = Array.from(htmlCollection);

// Method 2 (spread)
const arr = [...htmlCollection];
```

Note: NodeList from `getElementsByName()` supports `forEach()` directly without conversion.

JavaScript Output Operations

Method	Description
`alert(msg)`	Popup message box, returns undefined
`confirm(msg)`	OK/Cancel dialog, returns boolean
`document.write()`	Write HTML to page (use before page load)
`innerHTML`	Adds element as child to container
`outerHTML`	Replaces the element itself
`innerText`	Visible text only (respects CSS visibility)
`textContent`	All text including hidden elements
`console.log/warn/error/info`	Developer console output

innerHTML vs outerHTML:
`innerHTML` adds as a child inside the target.
`outerHTML` replaces the target element entirely.

innerText vs textContent:
Use `textContent` when you need all text (including hidden) and better performance.
Use `innerText` when you need visible text respecting CSS layout.

document.write() Limitation

Warning: If `document.write()` is called after the page finishes loading, it wipes out the entire existing document and rebuilds from scratch. Prefer `createElement()` + `appendChild()` instead.

```
// Safe alternative
const h1 = document.createElement("h1");
h1.textContent = 'Welcome to JavaScript';
document.body.appendChild(h1);
```

DAY 4

Output & Input Techniques

28 April 2025

JavaScript Input Mechanisms

1. **prompt()** — returns `string | null`
2. **Query String** — from URL parameters
3. **Form elements** — text, select, checkbox etc.

prompt()

- Returns `null` if user clicks Cancel or presses Escape.
- Returns an empty string `""` if OK is clicked without input.
- Returns the entered string otherwise.

```
const input = prompt('Enter the folder name');

if (input == null) {
    // User clicked Cancel
} else if (input.length == 0) {
    // User clicked OK without typing
} else {
    // Use input value
}
```

Query String

A Query String is a combination of **URL + query parameters (Key=Value)**.
Accessed via BOM: `location.search`
Query string data is received as a **string**.

```
const queryString = location.search;
console.log(queryString); // ?myName=sachin

// Extract value after '='
```

```
const value = queryString.slice(queryString.indexOf('=') + 1);
console.log(value); // sachin
```

DAY 6

var, let, const & Hoisting

05 May 2025

Hoisting

Hoisting is a phenomenon in JavaScript where variables and functions can be used before their declaration in the code.

If a `var` variable holds a function expression or arrow function, it **cannot** be used before initialization — doing so results in a `TypeError`.

var

- Supports hoisting with `undefined` as the default value
- Redeclaring the same variable is allowed
- Reassignment is allowed
- Function-scoped (also global-scoped when declared outside a function)
- Supports shadowing

let

- Supports hoisting — but variable lives in the **Temporal Dead Zone (TDZ)**
- Accessing a TDZ variable throws: `ReferenceError: Cannot access before initialization`
- Redeclaring is **not** allowed
- Reassignment is allowed
- Block-scoped
- Supports shadowing (redeclare in a different scope)

const

- Supports hoisting — lives in TDZ
- Redeclaring is **not** allowed
- Reassignment is **not** allowed (throws `TypeError`)
- Block-scoped
- Supports shadowing
- Must be initialized at the time of declaration

Comparison Table

Feature	var	let	const
Scope	Function/Global	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Shadowing	Yes	Yes	Yes

DAY 7

Variables, Datatypes & JSON

06 May 2025

Variable Naming Rules

1. Must start with an alphabet, `_`, or `$`
2. `_variableName` → signals the variable is not yet fully implemented (protected/partial-private)
3. Avoid special characters (ECMA standard)
4. Can be alphanumeric: e.g., `product2020`
5. Names are case-sensitive
6. Maximum 255 characters
7. Avoid single-character and overly long names
8. Do not use reserved keywords (45 total)
9. Always use **camelCase** and choose meaningful names

Primitive vs Non-Primitive Types

Property	Primitive	Non-Primitive
Memory	Stack (LIFO)	Heap
Size/Range	Fixed	Variable
Mutability	Immutable	Mutable
Examples	number, string, boolean, null, undefined, BigInt, Symbol	Array, Object, Map, Date, Function

JavaScript Object vs JSON

```
// JavaScript Object
let user = { id: 10, name: 'sachin', isMarried: true };

// Convert JS Object → JSON string
let jsonUser = JSON.stringify(user);
// {"id":10,"name":"sachin","isMarried":true}

// Convert JSON string → JS Object
```

```
let myData = '{"country":"IND","username":"Nitin"}';
let jsObj = JSON.parse(myData);
```

DAY 8

Number Type & Conversions

07 May 2025

Number Types in JavaScript

Type	Example
Signed Integer	`-5`
Unsigned Integer	`5`
Floating Point	`34.30`
Exponent	`2e3` → 2000
BigInt	`23283928354748n`
Hexadecimal	`0x0033`
Octal	`0o748`
Binary	`0b1010`

IEEE 754 Standard

JavaScript uses a single **Number type** following the **IEEE 754 64-bit floating-point standard**.

- 1 bit — MSB (sign: + or -)
- 11 bits — Exponent (holds decimal)
- 52 bits — Mantissa (holds data)

```
console.log(0.1 + 0.2);           // 0.30000000000000004 (precision issue)
console.log((0.1 + 0.2).toFixed(2)); // "0.30" — solution

console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

Number Conversions

```
Number("42")    // 42
Number("abc")   // NaN
parseInt("42")  // 42
parseFloat("3.14") // 3.14
```

```
Number.isNaN(NaN)  // true

let cost = 45000;
cost.toString();  // "45000"

let options = { style: 'currency', currency: 'GBP' };
cost.toLocaleString('en-GB', options); // "£45,000.00"
```

DAY 9 & 10

Template Literals & Strings

12 May 2025

Template Literals

Template literals use backticks (`) and support:

- **Data binding:** `${variableName}`
- **Multi-line strings**
- **Expressions:** `${age > 18 ? 'Major' : 'Minor'}`
- **Tagged templates**

```
let userName = 'Dhoni';
let userAge = 42;
document.write(`Hello ${userName}, Status: ${userAge > 18 ? 'Major' : 'Minor'}`);

// Tagged template
function printByHighlighting(strings, value) {
    return `${strings[0]} <b>${value}</b> ${strings[1]}`;
}
let name = 'Javascript';
document.write(printByHighlighting`Learn ${name} from PWIOI`);
```

Binding Types

Binding Type	Description	Example
Data Binding	Bind variable to output	`${variableName}`
Style Binding	Attach inline CSS dynamically	`element.style.cssText = '...'`
Class Binding	Attach CSS class to element	`element.className = 'myClass'`

DAY 11

String Methods – Part 1

13 May 2025

String Methods Reference

Method	Returns	Description
`length`	number	Character count
`toUpperCase()`	string	Converts to uppercase
`toLowerCase()`	string	Converts to lowercase
`startsWith(str)`	boolean	Checks beginning
`endsWith(str)`	boolean	Checks ending
`charAt(n)`	string	Character at index n
`charCodeAt(n)`	number	ASCII code at index n
`indexOf(str)`	number	First occurrence index, -1 if not found
`slice(start, end)`	string	Extract substring (supports negative)
`substring(start, end)`	string	Extract substring (no negative)
`trim()`	string	Remove leading/trailing spaces
`split(delim, limit)`	string[]	Split into array
`match(regEx)`	array | null	RegEx match

startsWith() and endsWith()

```
let str = 'sachin@gmail.com';
str.endsWith('@gmail.com');  // true
str.endsWith('@yahoo.com');  // false

let str2 = 'IPL2025';
str2.startsWith('IPL');      // true
str2.startsWith('ipl');      // false  (case-sensitive)
```

charAt() and charCodeAt()

ASCII ranges: a–z: 97–122 | A–Z: 65–90

> `charCodeAt()` treats null/undefined/NaN index as 0.

```
let str = 'sachin';
str.charAt(0);      // 's'
str.charCodeAt(0);  // 115

// Check if starts with uppercase
let flag = str.charCodeAt(0) >= 65 && str.charCodeAt(0) <= 90;
```

slice() vs substring()

Feature	slice()	substring()
Negative index	Supported (counts from end)	Treated as 0
start > end	Returns empty string	Arguments are swapped
NaN argument	Treated as 0	Treated as 0

```
let str = 'sachinrameshtendulkar';
str.slice(6, 16);       // "rameshtend"
str.slice(-9, -6);      // "ten"
str.substring(7, 15);   // "ameshtend"
str.substring(7, 0);    // "sachinr" (args swapped)
```

DAY 12

String Methods – Part 2

15 May 2025

split()

Splits a string based on a delimiter. Accepts two arguments: **delimiter** (string/regex) and optional **limit** (number).

```
let names = 'sachin,saurav,dravid,yuvraj,dhoni';
names.split(',');      // ['sachin','saurav','dravid','yuvraj','dhoni']
names.split(',', 4);   // ['sachin','saurav','dravid','yuvraj']
```

Corner Cases

Expression	Result
`"".split()`	`['']`
`"".split('')`	`[]`
`'hello'.split()`	`['hello']`
`'hello'.split('', 0)`	`[]`
`'P,W,,'.split(',')`	`['P','W','','']`

trim()

```
let str = '   sachin ramesh tendulkar  ';
console.log(str.length);          // 28
console.log(str.trim().length);   // 23
```

Truthy and Falsy Values

Falsy	Truthy
`""` , `0` , `null` , `undefined` , `NaN` , `0n` , `-0` , `false`	`" "` , any number ≠ 0, `[]` , `{}` , functions, non-empty strings

DAY 13

match(), Boolean & Operators

19 May 2025

match(regEx)

Returns an array if string matches the regex, otherwise returns `null`.

```
let mobile = '+919876543210';
let regEx = /^\+91\d{10}$/;
mobile.match(regEx);   // array (match found)

let bad = '99988877766';
bad.match(regEx);      // null
```

Always use `^` and `$` to ensure matching from start to end. Without them, partial matches succeed.

Comparison Operators

Operator	Description
`==`	Loose equality — performs type conversion
`===`	Strict equality — no type conversion
`!=`	Loose not-equal
`!==`	Strict not-equal

```
2 == '2'    // true  (Number('2') → 2)
2 === '2'   // false (different types)

null == undefined   // true
null === undefined  // false
NaN === NaN         // false  (NaN is never equal to itself)
```

Ternary Operator

```
// Syntax
condition ? valueIfTrue : valueIfFalse;
```

```
// Example
let status = checkBox.checked ? 'In Stock' : 'Out of Stock';

// Toggle button class
btn.className = btn.className === 'bi bi-sort-alpha-down'
  ? 'bi bi-sort-alpha-up'
  : 'bi bi-sort-alpha-down';
```

+ Operator Rules

Expression	Result	Reason
`5 + 10`	15 (number)	Both are numbers
`"3" + 2`	"32" (string)	One operand is string → concatenation
`5 + true`	6 (number)	true → 1
`"Result: " + 5 + 10`	"Result: 510"	Left to right, string first
`5 + 10 + " result"`	"15 result"	Numbers add first, then concat

DAY 14

Conditional Statements

20 May 2025

if Statement

```
if (condition) {
    // executes when condition is true
}
```

if-else Statement

```
if (condition) {
    // true block
} else {
    // false block (no condition needed)
}
```

Nested if-else

```
if (condition1) {
    if (condition2) {
        if (condition3) {
            // all true
        } else { /* condition3 false */ }
    } else { /* condition2 false */ }
} else { /* condition1 false */ }
```

else-if Ladder

```
if (condition1) {
    // block 1
} else if (condition2) {
    // block 2
} else if (condition3) {
    // block 3
} else {
```

```
    // default block
}
```

DAY 15

switch / case / default

29 May 2025

switch Statement

Use `switch` when a set of statements executes based on a **predefined condition value**.

```
switch (type) {
    case label1:
        // statements
        break;
    case label2:
        // statements
        break;
    default:
        // statements
        break;
}
```

Key Rules

1. The `type` is compared with `label` using **strict equality** (`===`).
2. `default` can be placed anywhere in the switch hierarchy.
3. Without `break` , **fall-through** occurs — execution continues into the next case.
4. Use fall-through deliberately when multiple cases should execute the same block.
5. Only one `default` is allowed.
6. Duplicate case labels are permitted.
7. `switch` argument can be of any type: boolean, string, number, object, function.
8. `return` can be used as a jump statement instead of `break` .

DAY 17

Non-Primitive Types & Arrays

30 May 2025

Non-Primitive Characteristics

- Memory allocated in the **Heap**
- Structure and range are **not predefined**
- **Mutable** — changes reflect in the same memory location

Array

Used to store multiple values under a single variable name. Memory is stored as **Key:Value** pairs (not necessarily contiguous).

```
// Static array
let arr = ['sachin', 51, 53.4, true];

// Dynamic array
let arr2 = new Array(5);
```

Reading Elements

```
// a. for loop
for (let i = 0; i < arr.length; i++) { console.log(arr[i]); }

// b. for...in (iterates over keys/indices)
for (const key in arr) { console.log(arr[key]); }

// c. for...of (iterates over values directly)
for (const element of arr) { console.log(element); }
```

Array Destructuring

```
const [myName, myAge, myAvg, married] = ['sachin', 51, 53.4, true];

// Skip elements
const [name, , gender] = ['sachin', 51, 'M'];

// Default values
```

```
const [a, b = 'Not Mentioned'] = ['sachin'];

// Swap without temp variable
[a, b] = [b, a];
```

Object Destructuring

```
const user = { userName: 'sachin', age: 51, gender: 'M' };

// Basic
const { userName, age } = user;

// Alias
const { userName: playerName, age: playerAge } = user;

// Default value
const { userName, salary = 'Not disclosed' } = user;
```

Rest Operator (...)

Usage Context	Example	Effect
Function Parameters	`function fn(...args) {}`	Collects all arguments into array
Array Destructuring	`[a, b, ...rest] = [1,2,3,4]`	Rest in an array
Object Destructuring	`const {x, ...rest} = obj`	Rest as object

Rest Operator Rules:
1. Only one rest parameter per function.
2. Rest parameter must be the **last** parameter.
3. Cannot have rest before normal params.

DAY 18

Array Methods – map() & forEach()

03 June 2025

map()

Transforms each element and creates a **new array**. Does not modify the original array.

```
// Syntax
array.map(function(element, index, array) {
    return newValue;
});

// Example
let arr = [1, 2, 3, 4];
let doubled = arr.map(value => value * 2); // [2, 4, 6, 8]

// With objects
let users = [{ userName: 'sachin', age: 51 }, { userName: 'kohli', age: 38 }];
let names = users.map(user => user.userName); // ['sachin', 'kohli']
```

forEach()

Iterates over each element. **Does not return a value** (returns undefined). Used for side effects: logging, DOM manipulation.

```
array.forEach(function(element, index, array) {
    // do something
});

let items = ['RCB', 'Punjab', 'MI', 'GT'];
items.forEach((item, index) => {
    console.log(`${index + 1}. ${item}`);
});
```

map() vs forEach()

Feature	map()	forEach()
Returns	New array	undefined

Modifies original	No	Only if explicitly done
Use case	Transform data	Side effects
Chainable	Yes	No

DAY 19

Array – Insert, Delete & Sort

04 June 2025

Insertion Methods

Method	Returns	Description
`push(...items)`	number (new length)	Insert at end
`unshift(...items)`	number (new length)	Insert at beginning
`splice(start, 0, ...items)`	[]	Insert at specified index
`concat(...items)`	New array	Returns new merged array

```
let arr = ['sachin', 'saurav', 'yuvi'];
arr.push('kohli', 'dhoni');         // Insert at end
arr.unshift('Gavaskar', 'KapilDev'); // Insert at front
arr.splice(3, 0, 'SRH', 'KKR');    // Insert at index 3
```

Deletion Methods

Method	Returns	Description
`pop()`	removed element	Remove from end
`shift()`	removed element	Remove from front
`splice(start, count)`	deleted elements[]	Remove at specified position

Sorting

`sort()` defaults to **string (Unicode) sorting**, which can produce incorrect results for numbers.
Use a **custom comparator** for numeric sorting.

```
// String sort (default)
['apple', 'cherry', 'banana'].sort(); // ['apple', 'banana', 'cherry']

// Numeric ascending
[10, 5, 2].sort((a, b) => a - b); // [2, 5, 10]

// Numeric descending
```

```
[10, 5, 2].sort((a, b) => b - a); // [10, 5, 2]

// Rule:
// (a - b) < 0  → a comes before b
// (a - b) > 0  → b comes before a
// (a - b) === 0 → keep original order
```

DAY 20

filter() and reduce()

05 June 2025

filter()

Returns a **new array** containing only elements that pass the test (callback returns `true`).

```
// Syntax
array.filter(callback(element, index, array));

// Examples
let names = ['sachin', 'saurav', 'kohli', 'dhoni', 'yuvi'];
let longNames = names.filter(name => name.length > 5);
// ['sachin', 'saurav', 'kohli']

let cart = [
    { id: 1, name: 'Laptop', price: 50000, available: true },
    { id: 2, name: 'Mouse',  price: 500,   available: false },
    { id: 3, name: 'Keyboard', price: 1000, available: true }
];
let available = cart.filter(item => item.available === true);
```

reduce()

Reduces the array to a **single value** by accumulating a result.

```
// Syntax
array.reduce(callback(accumulator, currentValue, index, array), initialValue);

// Sum of numbers
let nums = [1, 2, 3, 4];
let total = nums.reduce((total, data) => total + data, 0); // 10

// Chaining filter + reduce
let amountToPay = cart
    .filter(item => item.available)
    .reduce((total, obj) => total + obj.price, 0);

// Count occurrences
let iplTeams = ['CSK', 'MI', 'CSK', 'MI', 'SRH', 'RCB', 'KKR', 'KKR', 'MI', 'CSK'];
```

```
let result = iplTeams.reduce(function(obj, team) {
    obj[team] = (obj[team] | 0) + 1;
    return obj;
}, {});
```

Arrow Function Rules

1. No `function` keyword
2. Single parameter → no parentheses needed
3. Single statement body → no curly braces needed
4. Implicit return when no curly braces

DAY 21

Asynchronous JavaScript – fetch(), async/await

06 & 09 June 2025

Synchronous vs Asynchronous

JavaScript is single-threaded and synchronous by default — it executes one line at a time.

For async operations, it uses **Web APIs / Browser APIs**: `setTimeout()` , `fetch()` , Promises, async/await.

Component	Role
Call Stack	Main thread — executes JS code line by line
Web API	Browser features: setTimeout, fetch, DOM Events
Callback Queue	Holds callbacks after async task completes
Event Loop	Moves callbacks to call stack when it's empty
Microtask Queue	Holds .then() and .catch() (higher priority)

Promise

A Promise has 3 phases: **Pending** → **Resolved** (fulfilled) or **Rejected**.

```
let promise = new Promise(function(resolve, reject) {
    if (condition) {
        resolve(data);
    } else {
        reject(error);
    }
});

promise
    .then(function(data) { /* success */ })
    .catch(function(error) { /* failure */ })
    .finally(function() { /* always runs */ });
```

async / await

Syntactic sugar for Promises — makes async code look synchronous.

```
async function loadData() {
    try {
        const URL = '../data/products.json';
        let response = await fetch(URL);
        let products = await response.json();

        products.forEach(product => {
            // render to DOM
        });
    } catch (error) {
        console.log("Error:", error);
    }
}
```

async — marks a function to execute asynchronously.
await — pauses execution until the Promise resolves.

DAY 22

Objects & API Integration

11 June 2025

Object

An object has two parts:
has-part — properties (data)
does-part — methods (logic)

```
let obj = { 'name': 'sachin', 'age': 51, isMarried: true, Salary: 45456.756 };

// Reading
console.log(obj.name);        // dot operator (when key is known)
console.log(obj['Salary']);   // bracket operator (for dynamic/non-standard keys)
```

Object Utility Methods

Method	Returns	Description
`Object.keys(obj)`	string[]	All keys
`Object.values(obj)`	any[]	All values
`Object.entries(obj)`	[key, value][]	Key-value pairs

```
// Check key existence
if ('Salary' in user) { console.log(user['Salary']); }

// Delete a key
delete user.isMarried;

// Iterate keys with types
Object.keys(user).forEach(key => {
    console.log(`${key} → ${user[key]} → ${typeof user[key]}`);
});
```

When to use dot vs bracket notation

Situation	Use
Key is a valid identifier, known ahead of time	`obj.key`

Key is in a variable	`obj[keyVariable]`
Key has special characters or starts with a number	`obj['first-name']`

Object with Methods — `this` keyword

Important: When a function is used as a value inside an object, use `this` to access the object's properties. Arrow functions do NOT have their own `this` — they inherit from the outer scope.

```
var product = {
    'ProductName': 'Samsung TV',
    'Quantity': 2,
    'Price': 35000,
    'Total': function() {
        return this.Quantity * this.Price;  // 'this' refers to product
    }
};
```

DAY 24

Map, Symbol & Date

18 June 2025

Map

An alternative to Object. Key can be **any type**. Insertion order is preserved.

Method/Property	Description
`set(K, V)`	Add a key-value pair
`get(K)`	Get value by key
`has(K)`	Check if key exists
`delete(K)`	Remove a key
`clear()`	Remove all entries
`keys()`	All keys
`values()`	All values
`entries()`	All key-value pairs
`size`	Total count of entries

```
let map = new Map();
map.set('name', 'sachin');
map.set(10, 'ID');
map.set(true, 'IsMarried');

for (const [key, value] of map.entries()) {
    console.log(`${key}: ${value}`);
}
```

Use `for...of` to iterate a Map.

Object Limitations (why Map exists)

- Key must be of String type in Object
- No built-in size/count property
- Searching and deletion require extra operators

- Best suited only for structured (schema-defined) data

Symbol Type

A **primitive type** used for unique identification of keys in an object. Symbol keys are **hidden during iteration**.

```
let ID = Symbol();
let product = {
    [ID]: 10,
    name: 'Samsung TV',
    cost: 45000
};

// Symbol key NOT visible in for...in / Object.keys()
console.log(product[ID]);  // 10 — access individually

let id1 = Symbol('PW');
let id2 = Symbol('PW');
console.log(id1 === id2); // false — always unique
```

Date Type

```
let now = new Date();            // Current date and time
let mfd = new Date('2025-06-01'); // Specific date
let deptTime = new Date('2025-06-19 20:45:36');
```

Method	Returns	Notes
`getDate()`	1–31	Day of month
`getDay()`	0–6	0 = Sunday
`getMonth()`	0–11	0 = January
`getFullYear()`	4-digit year	
`getHours()`	0–23	
`getMinutes()`	0–59	
`getSeconds()`	0–59	
`toLocaleDateString()`	string	Locale-formatted date
`toLocaleTimeString()`	string	Locale-formatted time

DAY 25

setTimeout, setInterval & Debouncing

19 June 2025

Closure

A closure is a function that remembers the variables from its **lexical scope** even when executed outside that scope.

```
function a() {
    var i = 10;
    function b() {
        console.log(i); // b() has closure over i
    }
    b();
}
a();
```

setTimeout()

Runs a callback function **once** after a specified delay (in ms).

```
// Syntax
setTimeout(callbackFunction, delayInMs, arg1, arg2);

// Example
let id = setTimeout(function(name) {
    console.log(`Hello: ${name}`);
}, 2000, 'sachin');

clearTimeout(id); // Cancel before it fires
```

Feature	Detail
Purpose	Run function after a delay
Async/Sync	Asynchronous
Blocks execution?	No
Cancel it?	Yes — `clearTimeout(id)`

setInterval()

Repeats a callback at a fixed interval until stopped.

```
// Syntax
let id = setInterval(function, delayMs, param1, param2);

// Example – live clock
let id = setInterval(function() {
    document.querySelector('h1').textContent = new Date().toLocaleTimeString();
}, 1000);

clearInterval(id); // Stop the interval
```

Debouncing

Bouncing — function releases immediately into memory without locking.
Debouncing — function is locked in memory for a specific duration so other tasks can complete first.

Controlled using `setTimeout()` + `clearTimeout()`.

Classic Interview Question — var vs let in setTimeout

```
// Problem with var (prints 6 five times)
function x() {
    for (var i = 1; i <= 5; i++) {
        setTimeout(function() { console.log(i); }, i * 1000);
    }
}
// Reason: all callbacks share the same 'i' reference → by the time they run, i = 6

// Solution 1: Use let
for (let i = 1; i <= 5; i++) {
    setTimeout(function() { console.log(i); }, i * 1000);
}

// Solution 2: Use closure with var
for (var i = 1; i <= 5; i++) {
    function close(a) {
        setTimeout(function() { console.log(a); }, a * 1000);
    }
```

```
    close(i);
}
```

DAY 26

Functions, Callbacks & Promises

20 June 2025

Function Types

Type	Syntax
Function Declaration	`function name(params) { }`
Function Expression	`let name = function(params) { }`
Arrow Function	`let name = () => { }`
Anonymous (IIFE)	`(function() { })()`
Named Function Expression	`let fn = function hello() { }`

> Functions are **First Class Citizens** in JavaScript — they can be passed as arguments and returned from other functions.

Callback Function

A function whose body is passed as an argument to another function. The receiving function calls it when needed.
Can be used synchronously or asynchronously.

Exception Handling

Exception — a runtime mistake due to faulty user input causing abnormal termination.
Exception handling — promotes graceful termination.

```
try {
    // risky code
} catch (error) {
    console.log(error.name);    // e.g. TypeError
    console.log(error.message); // description
    console.log(error.stack);   // stack trace
} finally {
    // always executes — close resources
}
```

> All exceptions in JavaScript are **Unchecked Exceptions**.

Promise Phases

Phase	Meaning
Pending	Not yet started / in progress
Resolved	Successfully completed
Rejected	Failed

> `finally()` of a Promise executes regardless of resolve or reject.

DAY 27

Arrow Functions vs Function Expressions

26 June 2025

Key Differences

Feature	Function Expression	Arrow Function
`this`	Dynamic — depends on caller	Lexical — inherits from outer scope
`arguments` object	Available	Not available
Constructor	Can be used with `new`	Cannot be used as constructor
Syntax	Verbose	Concise

```
// this in function expression
const obj = {
    name: "Sachin",
    greet: function() { console.log("Hi " + this.name); } // "Hi Sachin"
};

// this in arrow function
const obj2 = {
    name: "Sachin",
    greet: () => { console.log("Hi " + this.name); } // "Hi undefined"
};
```

arguments Object

```
// Available in function expression
function printArgs() {
    console.log(arguments); // [1, 2, 3]
}
printArgs(1, 2, 3);

// Convert to real array
function printArgs() {
    let args = Array.from(arguments);
    console.log(args.map(x => x * 2));
}
```

```
// NOT available in arrow functions
const print = () => { console.log(arguments); }; // ReferenceError
```

DAY 28

Rest, Spread & Function Generators

27 June 2025

Spread Operator

Expands iterable elements (array, string, object) into individual elements.

```
// Spread array into function args
let data = [10, 'sachin', 53.5, true];
printData(...data);

// Clone array (shallow copy)
let arr = [1, 2, 3];
let clone = [...arr, 4]; // [1, 2, 3, 4]

// Clone object
let obj = { name: 'kohli', age: 37 };
let updated = { ...obj, name: 'viratKohli', isMarried: true };
```

Spread vs Rest

Operator	Used As	Effect
Rest `...`	Function parameter / destructuring	Packs multiple values into one variable
Spread `...`	Function argument / cloning	Expands one value into multiple elements

Shallow Copy vs Deep Copy

```
// Shallow Copy (spread)
let cloneObj = { ...obj };
cloneObj.name = 'kohli';       // primitive — original unchanged
cloneObj.Address.City = 'Delhi'; // non-primitive — ORIGINAL CHANGED (shared
reference)

// Deep Copy (structuredClone)
let deepClone = structuredClone(obj);
deepClone.Address.City = 'Delhi'; // original NOT affected
```

Function Generators

A special function that can **pause execution** and resume later using `yield`.

```
function* generateValues() {
    yield "sachin";
    yield "saurav";
    yield 'rahul';
    yield 'kohli';
}

let gen = generateValues();
let result = gen.next(); // { value: "sachin", done: false }

while (!result.done) {
    console.log(result.value);
    result = gen.next();
}
```

Keyword	Purpose
`function*`	Declares a generator function
`yield`	Pauses function and returns a value
`.next()`	Resumes and gets the next value

DAY 29

Higher Order Functions

30 June 2025

Higher Order Functions

A function that **accepts another function as a parameter** (or returns a function) is called a Higher Order Function.

```
const radius = [3, 4, 1, 2];

const area = (r) => (Math.PI * r * r).toFixed(2);
const circumference = (r) => (Math.PI * 2 * r).toFixed(2);
const diameter = (r) => 2 * r;

// Higher order function
function calculate(radius, logic) {
    let output = [];
    for (let i = 0; i < radius.length; i++) {
        output.push(logic(radius[i]));
    }
    return output;
}

console.log(calculate(radius, area));
console.log(calculate(radius, circumference));
console.log(calculate(radius, diameter));
```

Extending Built-in Classes with prototype

You can add any user-defined function to a built-in class using `prototype`.

```
// Syntax
ClassName.prototype.key = function() { };

// Example: add calculate() to Array
Array.prototype.calculate = function(logic) {
    let output = [];
    for (let i = 0; i < this.length; i++) {
        output.push(logic(this[i]));
    }
```

```
    return output;
};

console.log(radius.calculate(area));
```

DAY 33

Loops, Jump Statements & React Intro

04 July 2025

Looping Statements

Loop	When to use
`for`	When number of iterations is known
`while`	When iteration count is decided dynamically
`do...while`	When the block must execute at least once regardless of condition

Jump Statements

Statement	Effect
`break`	Exits the loop entirely
`continue`	Skips current iteration, jumps to next
`return`	Exits the function and returns to caller

return vs throw

Feature	return	throw (error)
Purpose	Give output to caller	Signal a problem occurred
Control Flow	Ends function, passes a value	Jumps to nearest catch block
Used in	Normal logic	Error handling

React — Introduction

React is a JavaScript library used to modify the DOM faster.

Core API:
`React.createElement("tag", {}, ...children)` → ReactElement (Virtual DOM / JS object)
`ReactDOM.createRoot(element).render(heading)` → renders JS object → HTML element → Actual DOM

Parcel

Parcel is a bundler used to build and push projects to a Production environment.

Dependency Type	Purpose	Install Command
Developer Dependency	Used only in development	`npm i -D parcel`
Production Dependency	Used in testing & production	`npm i package`

Package version format: `Major.Minor.Patch`

DAY 34

Special Operators & instanceof

10 July 2025

Operators Reference

Operator	Purpose
`void`	Function with no return value → returns `undefined`
`new`	Create instance of a class
`in`	Check key existence in object
`of`	Iterate values in iterable
`typeof`	Returns type as string
`instanceof`	Check if object is instance of a class
`delete`	Delete a property from an object
`yield`	Pause generator function

instanceof

Returns `true` if the reference is of the same type, child type, or Object type.

```
class Animal { }
class Dog extends Animal { }

let a = new Dog();
console.log(a instanceof Animal); // true
console.log(a instanceof Dog);    // true
console.log(a instanceof Object); // true
console.log(a instanceof Array);  // false
console.log(a instanceof String); // false
```

DAY 35

OOP – Classes & Constructors

10 & 15 July 2025

What is a Class?

A **class** is a program template that provides logic and data which can be implemented and customized according to requirements.

- Behaves as an **Entity** when it has business interactions
- Behaves as a **Model** when it handles data interactions
- Used as a blueprint to create multiple objects

```
class ClassName {
    // properties (data)
    // constructor
    // setter/getter
    // methods
}
```

Constructor

An **anonymous method** called automatically when an object is created.
Used as an **initializer block**. Uses `this` to avoid shadowing when parameter names match properties.

```
class Student {
    constructor(name, age, email) {
        this.name = name;
        this.age = age;
        this.email = email;
    }
    printData() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}
let std = new Student('sachin', 53, 'sachin@gmail.com');
std.printData();
```

> JavaScript does NOT support constructor overloading. Use default parameters instead: `constructor(name, price = 0) { }`

Variable Accessibility

Syntax	Access Level
`variableName`	Public
`_variableName`	Protected / Partial-private (convention only)
`#variableName`	Private (enforced by JS engine)

Setter and Getter

```
class Product {
    _productName;

    set productName(newName) {
        if (role === 'admin') {
            this._productName = newName;
        }
    }

    get productName() {
        return this._productName;
    }
}

let p = new Product();
p.productName = 'Fossil';  // calls setter
console.log(p.productName); // calls getter
```

Modules

Export Type	Export Syntax	Import Syntax
Named Export	`export function sum() { }`	`import { sum as add } from './App.js'`
Default Export	`export default PI`	`import PI from './App.js'`
Both Combined	—	`import PI, { sum as add } from './App.js'`

DAY 36

Code Reusability – Aggregation & Inheritance

16 July 2025

Code Reusability vs Extensibility

Concept	Definition
Code Reusability	Accessing code from one module and using it in another
Code Extensibility	Extending a module without disturbing existing features

Achieved via: **a. Aggregation (HAS-A)** and **b. Inheritance (IS-A)**

Aggregation (HAS-A)

Accessing members of one class in another class **without a formal relationship**. Uses object-to-object communication — creates an instance of the other class.

```
class HDFCVersion2 {
    loan = 'House Loan, Education Loan';
    print() {
        let obj = new HDFCVersion1(); // HAS-A (Aggregation)
        obj.print();
        document.write(`${this.loan}`);
    }
}
```

Inheritance (IS-A)

Configures a formal relationship between classes. Uses `extends`. Child class automatically gets all parent features.
Use `super` to access parent class members and constructors.

```
class HDFCVersion2 extends HDFCVersion1 {
    loan = 'House Loan, Education Loan';
    print() {
        super.print(); // call parent method
        document.write(`${this.loan}`);
    }
}
```

```
// Constructor chain
class Child extends Parent {
    constructor(name, age, gender, profession) {
        super(name, age, gender); // must call super() first
        this.profession = profession;
    }
}
```

Important: In JavaScript, calling the parent constructor from a child class is **explicit** (must call `super()`). In many other OOP languages, it is implicit.

classList Property

Method	Description
`classList.add('cls')`	Add a class
`classList.remove('cls')`	Remove a class (ignored if not present)
`classList.contains('cls')`	Returns boolean
`classList.toggle('cls')`	Add if absent, remove if present
`classList.replace('old','new')`	Replace one class with another

Why use classList? Directly setting `element.className` overrides all existing classes. Use `classList` to safely add/remove individual classes without affecting others.

DAY 37

JavaScript Browser Events

18 July 2025

What is an Event?

An **event** is a message sent by a **sender** to its **subscriber** to notify of a change.
Events follow the **Observer design pattern** and use a **function pointer mechanism (Delegate)**.

Term	Meaning
Subscriber	Function that defines the actions to perform
Sender	Element that triggers the action (e.g., button's onclick)
Event Handler	The function assigned to an event
Event Listener	`addEventListener()` — configures events dynamically

Event Arguments

Every event handler has 2 default arguments: `this` and `event`

```
<button onclick="handleClick(this, event)">Pay</button>

function handleClick(obj, e) {
    // obj → info about the element (id, name, class, value...)
    // e   → info about the event (clientX, clientY, shiftKey, ctrlKey...)
    console.log(e.clientX, e.clientY, e.ctrlKey);
    console.log(obj.id, obj.name, obj.className);
}
```

Argument order is flexible — `insert(event, this)` and `insert(this, event)` both work.
You can also pass specific properties: `insert(this.id)`.

Event Categories

Category	Events
Mouse Events	onmouseover, onmouseout, onmousedown, onmouseup, onmousemove

Keyboard Events	onkeyup, onkeydown, onkeypress (deprecated)
Button Events	onclick, ondblclick, oncontextmenu, onselectstart
Timer Events	setTimeout, setInterval
Form Events	onsubmit, onreset
Element State Events	onblur, onchange, onfocus
Clipboard Events	oncut, oncopy, onpaste

DAY 38

Mouse, Keyboard & Clipboard Events

23 July 2025

Mouse Events

Event	Fires When
`onmouseover`	Mouse cursor hovers over the element
`onmouseout`	Mouse cursor leaves the element
`onmousedown`	Mouse button held down on the element
`onmouseup`	Mouse button released on the element
`onmousemove`	Mouse pointer moves on the element
`ondblclick`	Element is double-clicked
`oncontextmenu`	Right-click on the element
`onselectstart`	User starts selecting/dragging text

```
// Disable right-click
document.oncontextmenu = function() { return false; };

// Disable text selection
document.onselectstart = function() { return false; };

// Open image on double-click
document.querySelector("img").addEventListener("dblclick", function() {
    window.open("../images/photo.jpg", "_blank", "height=500 width=500");
});
```

Keyboard Events

Event	Use Case
`onkeydown`	Key is pressed down
`onkeyup`	Key is released
`onkeypress`	Deprecated — used for character codes

> Use `onkeyup` / `onkeydown` for character detection.
> Use `onkeypress` for character codes (keyCode, charCode, which).

Key event arguments: `keyCode`, `charCode`, `which`, `shiftKey`, `ctrlKey`, `altKey`

Element State Events

Event	Fires When
`onfocus`	Element receives focus (user clicks/tabs into it)
`onblur`	Element loses focus
`onchange`	Value of element changes

Clipboard Events

The **Clipboard** is the temporary memory in Windows for cut/copied content.

Event	Fires When
`oncopy`	User copies content
`oncut`	User cuts content
`onpaste`	User pastes content

```
// Prevent paste in password field
<input type="password" onpaste="return false;">
```

DAY 39

Form Events & Interactive Quiz Project

24 July 2025

Form Events

Event	Configured On	Fires When
`onsubmit`	`<form>`	User submits the form
`onreset`	`<form>`	User resets the form

```
<form action="" onsubmit="alert('form submitted')" onreset="alert('form cleared')">
    <button type="submit">Submit</button>
    <button type="reset">Reset</button>
</form>
```

Project: Interactive Quiz App

A timer-based quiz app using form events, setInterval, and DOM manipulation.

Key Features

- Loads questions one at a time from a data array
- 15-second countdown timer per question (auto-submits on timeout)
- Tracks score across all questions
- Shows final score when all questions are answered

Core Logic

```
const quizData = [
    { question: "What is the capital of France?",
      options: ["London", "Berlin", "Paris", "Madrid"],
      answer: "Paris" },
    { question: "What is 2 + 2?",
      options: ["3", "4", "22", "5"],
      answer: "4" },
    { question: "Which language runs in a browser?",
      options: ["Java", "C", "Python", "JavaScript"],
      answer: "JavaScript" }
];

let currentQuestion = 0;
let score = 0;
```

```
let timer;
let timeLimit = 15;

function loadQuestion() {
    clearInterval(timer);
    const current = quizData[currentQuestion];
    questionEl.textContent = current.question;
    optionsEl.innerHTML = '';
    current.options.forEach(option => {
        const label = document.createElement('label');
        label.innerHTML = `<input type="radio" name="option" value="${option}"
required>${option}`;
        optionsEl.appendChild(label);
    });
    startTimer(timeLimit);
}

function startTimer(seconds) {
    timerEl.textContent = `${seconds}s`;
    timer = setInterval(() => {
        seconds--;
        timerEl.textContent = `${seconds}s`;
        if (seconds === 0) {
            clearInterval(timer);
            submitAnswer();
        }
    }, 1000);
}

function submitAnswer() {
    const selected = quizForm.option.value;
    const correct = quizData[currentQuestion].answer;
    if (selected === correct) score++;
    currentQuestion++;
    if (currentQuestion < quizData.length) {
        loadQuestion();
    } else {
        showScore();
    }
}

quizForm.addEventListener('submit', function(e) {
    e.preventDefault();
    submitAnswer();
});

function showScore() {
    questionEl.style.display = 'none';
    optionsEl.style.display = 'none';
    quizForm.querySelector('button').style.display = 'none';
    timerEl.style.display = 'none';
    scoreEl.textContent = `You scored ${score} out of ${quizData.length}`;
}
```

```
loadQuestion();
```

DAY 98

BOM – Browser Object Model

Advanced Session

BOM Overview

The **Browser Object Model (BOM)** provides objects to interact with the browser window beyond the document.

Object	Purpose
`window`	Control the browser window
`location`	Get/set URL and navigation details
`history`	Manage browsing history
`navigator`	Get browser and client details
`document`	Handle elements inside the body

window Object

Member	Description
`prompt()`	Popup input box
`alert()`	Popup message box
`open()`	Open any file with specified features
`close()`	Close the window
`print()`	Invoke printer properties

location Object

Provides properties and methods to know and change the client's location/URL.

Property/Method	Description
`host`	Server name or IP address
`protocol`	Protocol: http, https, file
`port`	Port number

`pathname`	Current file path
`href`	Gets and sets the full URL
`search`	Returns the query string
`hash`	Returns current hash location

Q: How to navigate dynamically in JavaScript?

Use `location.href = 'newPage.html'`

```
// Dynamic navigation example
if (username === "nitin" && password === "nitin") {
    location.href = '../netflix-code.html';
} else {
    location.href = 'error-code.html';
}
```

navigator Object

Provides properties and methods to get client browser details.

Member	Description
`appName`	Browser family name
`appVersion`	Browser version
`userAgent`	Browser supported platforms
`platform`	Current platform
`language`	Browser language
`cookieEnabled`	Cookie status
`plugins[]`	All installed plugins
`mimeTypes[]`	All supported MIME types
`geolocation`	Get current geolocation

```
// Geolocation
navigator.geolocation.getCurrentPosition(function(position) {
    document.write(`
        Latitude: ${position.coords.latitude}
        Longitude: ${position.coords.longitude}
    `);
});

// Plugins
```

```
for (const plugin of navigator.plugins) {
    document.write(plugin.name + "<br>");
}
```

history Object

Member	Description
`length`	Total count of pages in current browsing history
`forward()`	Move to next page in browsing history
`back()`	Move to previous page in history
`go(n)`	Move to specific page in history (+ forward, - backward)

document Object

Member Type	Members
Methods	`getElementById()`, `querySelector()`, `querySelectorAll()`, `write()`, `writeln()`
Properties	`body`, `documentElement`, `head`, `title`, `forms`, `links`, `images`, `scripts`, `cookie`

✓

Notes Complete

All 41 sessions covered — from JavaScript fundamentals to BOM, OOP, Async, and DOM Events.

JavaScript Complete Class Notes | Days 1 – 98